

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I T Sanhith(192311228) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Case Study

1. A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.

ANS:

Analysis of Using MD5 for Password Storage and a Secure SHA-Based Approach

MD5 is a cryptographic hash function that produces a fixed-length 128-bit hash value. In the given scenario, the software company uses MD5 to store user passwords. After a security breach, attackers successfully crack multiple passwords. This occurs because MD5 is no longer considered secure for password protection.

Weaknesses of MD5

The major weakness of MD5 is its vulnerability to **collision attacks**. A collision occurs when two different inputs produce the same hash value. Researchers have demonstrated practical collision attacks against MD5, making it unsuitable for modern security applications.

Another important problem is that MD5 is **extremely fast**. Although speed may be useful for general hashing, it is a disadvantage for password storage. Attackers can perform billions of password guesses per second using modern GPUs or specialized hardware.

If passwords are stored using only:

MD5(password)

an attacker who obtains the database can use:

- Dictionary attacks
- Brute-force attacks
- Rainbow tables
- GPU-based password cracking

to recover weak or commonly used passwords.

For example, if a user's password is:

password123

the MD5 hash can be easily searched in precomputed databases or rainbow tables.

Proposed Secure Approach

A better approach is to use a modern SHA-based password hashing method such as **PBKDF2 with SHA-256 or SHA-512**. The password should also be combined with a unique random **salt**.

The process can be represented as:

Stored Hash = PBKDF2-HMAC-SHA256(Password, Salt, Iterations)

A salt is a random value generated separately for every user.

For example:

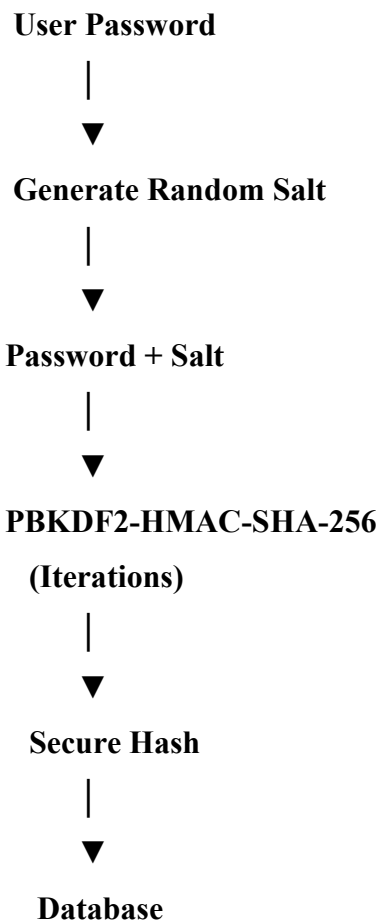
Password → Add Random Salt → PBKDF2-HMAC-SHA-256 → Stored Hash

The database should store:

- Username
- Salt
- Number of iterations
- Derived password hash

Using a salt prevents attackers from using the same precomputed rainbow table against all users.

Iterative hashing also increases the computational cost of every password guess.



2. A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.

ANS:

Digital signatures are used to provide:

- Authentication
- Integrity
- Non-repudiation

In the banking application, unauthorized transactions are being approved even though digital signatures are implemented. This indicates that the issue may not necessarily be with the mathematical concept of digital signatures itself, but with the **implementation, key management, or verification process**.

Possible Flaws

a) Poor Private Key Protection

A digital signature is generated using the user's private key:

$\text{Signature} = \text{Sign}(\text{Hash}(\text{Transaction}), \text{Private Key})$

If an attacker obtains the private key, they can create valid signatures for fraudulent transactions.

b) Improper Signature Verification

The bank may be verifying only the signature but not verifying the complete transaction data.

For example, suppose a user signs:

Transfer ₹1,000 to Account A

If an attacker changes the transaction after signing and the system does not correctly verify the complete transaction message, an unauthorized transaction could be approved.

The correct verification should ensure:

$\text{Verify}(\text{Public Key}, \text{Original Transaction Data}, \text{Signature}) = \text{Valid}$

The exact transaction details, including:

- Sender
- Receiver
- Amount
- Transaction ID
- Timestamp

must be included in the data that is signed.

c) Use of Weak Cryptographic Algorithms

The system may use outdated algorithms or insecure key sizes. Weak implementations may make signatures vulnerable to attacks.

d) Replay Attacks

An attacker may capture a previously valid signed transaction and submit it again if the banking system does not check transaction uniqueness.

e) Lack of Certificate Validation

If public keys are not properly associated with verified user identities, an attacker may substitute their own key.

3. An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.

ANS:

Kerberos is a network authentication protocol that uses a trusted Key Distribution Center (KDC) to authenticate users and services. It uses tickets and symmetric encryption to avoid repeatedly sending passwords over the network.

A replay attack occurs when an attacker captures a valid authentication message and retransmits it to gain unauthorized access.

A simplified attack scenario is:

1. A legitimate user authenticates with the Kerberos system.
2. The user receives a valid ticket.
3. The attacker captures a ticket or authenticator from the network or a compromised system.
4. The attacker retransmits the captured authentication information.
5. If the server does not properly validate timestamps, session validity, or uniqueness, the attacker may be granted access.

Proposed Enhancements

1. Use Timestamps

Kerberos authenticators contain timestamps. The server should verify whether the timestamp is within an acceptable time window.

For example:

Current Time - Authenticator Time $\leq$ Allowed Time Window

Old authentication requests should be rejected.

2. Implement a Replay Cache

The server should maintain a cache of recently used authenticators. If the same authenticator is received again, it should be rejected.

3. Use Short Ticket Lifetimes

Tickets should expire after a limited period. Short-lived tickets reduce the opportunity for attackers to reuse stolen credentials.

4. Synchronize System Clocks

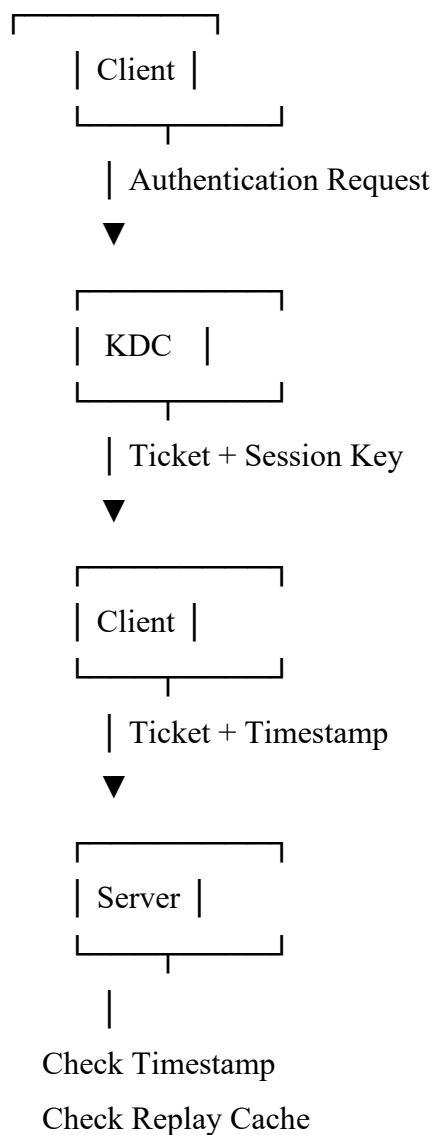
Kerberos depends heavily on timestamps. Client, server, and KDC clocks should be synchronized using a secure and reliable time synchronization mechanism.

5. Protect Session Keys

Session keys should be securely stored and removed from memory when no longer required.

6. Use Mutual Authentication

The client should also verify the identity of the server. This prevents attackers from impersonating legitimate services.



Check Ticket Validity



Access Granted

4. A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.

ANS:

A hash function is primarily designed to verify whether data has changed. However, a simple hash alone does not provide message authentication.

Suppose a sender transmits:

Message + SHA-256(Message)

For example:

M = "Transfer ₹10,000"

The sender calculates:

$H = \text{SHA-256}(M)$

and sends both the message and the hash.

An attacker can intercept the communication, modify the message, and calculate a new hash:

Modified Message + SHA-256(Modified Message)

The receiver cannot determine whether the message and hash were generated by the legitimate sender or by an attacker.

Therefore, a simple hash provides integrity only when the hash itself is obtained from a trusted source. It does not prove the identity of the sender.

Solution Using HMAC

HMAC stands for **Hash-based Message Authentication Code**. It combines a cryptographic hash function with a shared secret key.

The HMAC value is calculated as:

$\text{HMAC} = \text{HMAC-SHA256}(\text{Secret Key}, \text{Message})$

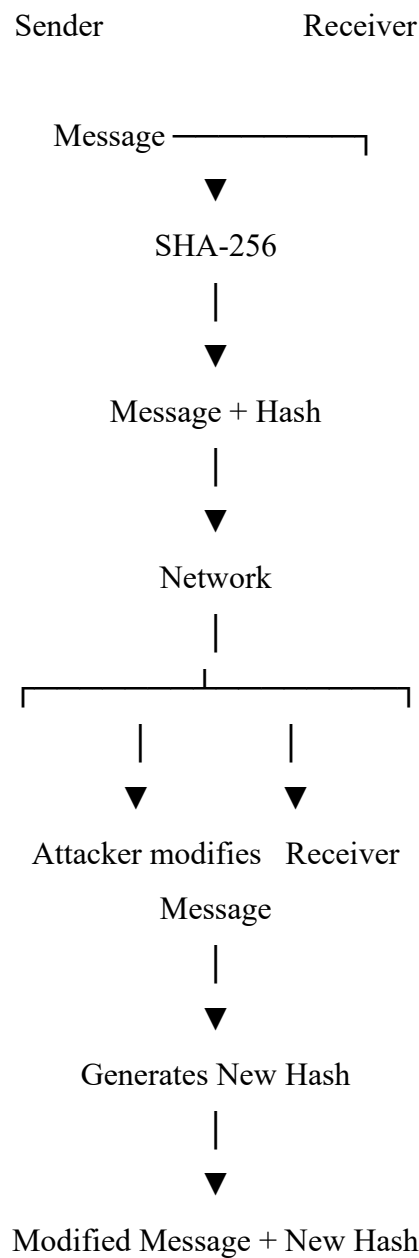
The sender and receiver both possess the secret key.

The process is:

Sender → Message + HMAC

The receiver calculates the HMAC again using the same secret key:
Expected HMAC = HMAC-SHA256(Secret Key, Received Message)
Then:
If Received HMAC = Expected HMAC → Accept
Otherwise:

SIMPLE HASH



✗ Receiver cannot identify the attacker

5. An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.

ANS:

X.509 certificates are used in secure communication protocols such as TLS/HTTPS. A certificate binds an identity, such as a domain or organization, to a public key.

Frequent trust validation errors indicate problems with certificate management, configuration, or the certificate chain.

Possible Issues

a) Expired Certificates

Every X.509 certificate has a validity period.

If:

Current Date > Certificate Expiry Date

the certificate is no longer trusted.

This can cause browsers and applications to display certificate warnings.

b) Incorrect Certificate Chain

A certificate is usually validated through a chain:

Server Certificate → Intermediate CA → Root CA

If the server does not provide the required intermediate certificate, clients may fail to establish a complete trust chain.

c) Untrusted Certificate Authority

If the certificate is issued by an unknown or untrusted Certificate Authority, the client's operating system or browser may reject it.

d) Domain Name Mismatch

The domain requested by the user must match the certificate's:

- Common Name (CN), or
- Subject Alternative Name (SAN)

For example, a certificate issued for:

www.bank.com

should not normally be used for:

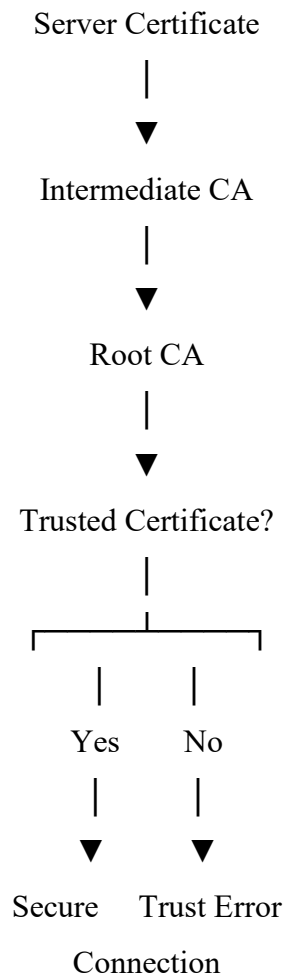
www.otherbank.com

e) Incorrect System Date and Time

Certificate validation depends on the current system time. If the client system clock is incorrect, a valid certificate may appear expired or not yet valid.

f) Certificate Revocation Problems

A certificate may have been revoked due to key compromise or other security issues. If revocation checking through CRL or OCSP is not configured correctly, trust validation may fail.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand